

SynGS: Synergizing Explicit Geometry and Transformer Depth Priors for Sparse-View 3D Gaussian Splatting

Junjie Geng,^{a,*} Ao Zhang,^a Lian Duan,^a

^aCommunication University of China, a State Key Laboratory of Media Convergence and Communication, Intelligent Network and Media Research Center, No. 1 Dingfuzhuang East Street, Chaoyang District, Beijing 100024, China

Abstract. In 360-degree scenes captured with sparse views, 3D Gaussian Splatting (3DGS) reconstruction is highly susceptible to the loss of edge detail, geometric instability in occluded regions, and overfitting to foreground regions. To mitigate these challenges under sparse-view conditions, we present SynGS, a 3D Gaussian Splatting method guided by geometric and depth priors for sparse-view 3D reconstruction. The proposed method leverages the visual hull to impose explicit geometric constraints and integrates a Transformer-based depth prior to facilitate global cross-view consistency, complemented by a specialized geometry enhancement pipeline. By combining reliable depth filtering with fine-grained structural detail recovery and incorporating a dynamic visibility regularization strategy, our approach enables adaptive reweighting of supervision signals during optimization. This framework enhances structural fidelity and boundary alignment while suppressing late-stage overfitting and floating artifacts. Empirical evaluations demonstrate that SynGS achieves competitive performance, particularly in PSNR and SSIM. On the Mip-NeRF 360 dataset, SynGS achieves gains of up to 3.56 dB in PSNR and 0.025 in SSIM under sparse-view settings. These results demonstrate the effectiveness of combining explicit geometric constraints with depth priors for stable sparse-view reconstruction.

Keywords: Novel View Synthesis, 3D Gaussian Splatting, Few-shot reconstruction, Geometry Prior-Guided Reconstruction, Vision Transformer.

*Junjie Geng, E-mail: gjj@cuc.edu.cn

1 Introduction

The reconstruction of 3D scenes from 2D imagery has long been a fundamental challenge in the fields of computer vision and computer graphics, aiming to recover both the geometric structure and surface appearance of a scene from limited image observations. This technology provides the requisite technical foundation for the digital modeling of complex real-world

environments. It holds transformative potential across diverse application domains, significant application value in domains including such as virtual reality, digital twins, autonomous driving, game development, cultural heritage preservation, and robotic navigation ^[1], providing crucial technical support for the digital modeling of complex real-world scenes. In practical deployment scenarios applications, however, due to acquisition costs, equipment setup constraints, and limited camera trajectories, it is often difficult to obtain multi-view images with sufficient coverage. Consequently, achieving high-fidelity 3D reconstruction from sparse viewpoints has emerged as a research endeavor of significant academic and practical importance. Therefore, 3D reconstruction from sparse views is of great research significance.

Compared with dense-view capture, sparse inputs provide only limited information about visible regions. As a result, 3D reconstruction methods tend to overfit local observations from sparse inputs, making it difficult to maintain global geometric consistency ^[2]. Furthermore, in such an underconstrained setting, sparse observations introduce greater geometric ambiguity, making occluded regions, textureless areas, and view boundaries more prone to missing geometry, structural distortions, and degraded appearance reconstruction quality ^[3]. Consequently, effectively estimating and completing the geometry and appearance of unseen regions, while establishing a stable initial geometric scaffold and reducing noise propagation from unreliable supervision, remains a critical challenge in sparse-view 3D reconstruction.

2 Related Work

Traditional 3D reconstruction methods typically rely on high-precision sensors such as multi-camera arrays or LiDAR to acquire dense geometric information^{[4][5]}. While these methods provide relatively accurate geometric priors, their high hardware costs, complex deployment procedures, and limited flexibility in practical application scenarios impose certain constraints.

In recent years, Neural Radiance Fields (NeRF)^[6] methods have proposed representing 3D scenes as continuous implicit radiance fields. This approach models 3D spatial coordinates and viewing directions using Multi-Layer Perceptron (MLP) networks to predict corresponding volume density and color, ultimately achieving high-fidelity geometry and appearance reconstruction through volume rendering. Consequently, such methods have become a primary research direction in 3D reconstruction. Under sparse-view conditions, several studies have introduced additional priors to

alleviate the instability caused by insufficient observations. For instance, Wang et al.^[7] incorporated pre-trained depth models to improve structural consistency, Deng et al.^[8] used SfM-estimated camera poses and point clouds as supervision, and Michael et al.^[9] regularized unobserved viewpoints to enhance cross-view consistency. Although these methods improve sparse-view reconstruction quality to some extent, NeRF-based approaches still rely on dense sampling and iterative optimization, resulting in high training and rendering costs, which limits their applicability in real-time scenarios.

To improve reconstruction efficiency, Kerbl et al. ^[10] introduced 3D Gaussian Splatting (3DGS), an explicit scene representation and rendering framework. This method represents scene geometry and appearance with differentiable 3D Gaussian ellipsoids, whose position, covariance, color, and opacity are jointly optimized. By adopting efficient tiled rasterization, 3DGS enables rapid training and real-time rendering ^[11]. Compared to implicit neural representation methods, the explicit representation and rasterization strategy of 3DGS offer significant advantages in efficiency, quickly making it a prominent research focus in sparse-view 3D reconstruction.

Building upon this framework, research has expanded into depth regularization, representation improvements, and generative priors. For example, Chung et al. ^[12] introduced depth estimation models aligned with SfM point clouds for depth regularization, adding smoothness constraints to enhance stability, though this may result in over-smoothed local details. Li et al. ^[13] implemented global-local normalization for soft and hard depths and replaced spherical harmonics with neural networks to mitigate overfitting; however, the effectiveness of this method decreases as the number of input images increases, and holes or gaps may still appear in certain scenes ^[14]. Zhu et al. ^[15] proposed real-time few-shot view synthesis using Gaussian splatting(FSGS), which utilizes initial sparse points from SfM and introduces a Pearson correlation loss to construct sparse Gaussian distributions that progressively fill the scene for few-shot novel view synthesis. Xiong et al. ^[16] enhanced sparse synthesis performance through depth map regularization and depth estimation models, using generative diffusion models and Score Distillation Sampling (SDS) to fill data gaps, while applying pruning strategies to optimize image quality. Overall, most existing 3DGS-based sparse reconstruction methods rely on depth information as supervision and typically employ predefined sampling strategies to initialize 3D Gaussians. While these approaches improve quality to an extent, they struggle to fully capture the diversity and intricate details of complex scenes.

Recently, other technical routes for sparse-view reconstruction have also emerged alongside NeRF and 3DGS. Huang et al.^[17] proposed SC-NeuS to extract multi-view constraints from neural surfaces, Zou et al.^[18] introduced Sparse3D by combining multi-view consistent diffusion models with category-based SDS, and Xiao et al.^[19] integrated implicit neural level sets into the SfM framework to improve sparse structural recovery. These methods enhance sparse-view reconstruction from different perspectives. However, under extreme sparsity, heavy noise, or unconstrained complex scenes, they still tend to rely on strong geometric priors, predefined models, or external supervision^{[20][21]}, which limits detail recovery, global consistency, and robustness.

3 Method

3.1 Overall Framework

We propose a 3D Gaussian Splatting (3DGS)-based reconstruction framework for sparse-view scenes that jointly incorporates geometric and depth priors. This approach is designed to strengthen the initialization constraints on Gaussian parameters, thereby improving the stability and geometric accuracy of the subsequent optimization process. The overall pipeline of the framework is illustrated in Fig. 1, where the inputs consist of multi-view images and their corresponding foreground masks, and the output is an optimized 3DGS scene representation.

Specifically, our method consists of three interconnected stages. In the first stage, we use multi-view silhouettes to construct a visual hull and generate a conservative yet stable visual hull point cloud, which serves as the geometric prior for initialization^[22]. This hull point cloud constrains the overall scene structure and mitigates geometric drift during the early stages of optimization under sparse-view settings. The second stage is a depth-guided densification pipeline. We introduce a depth prediction model as a depth prior and perform cross-view consistency filtering on the depth predictions from each view. Reliable regions are then back-projected to generate and densify point clouds. Building upon this, we employ voxel fusion to obtain an enhanced point cloud, which is explicitly mapped to 3DGS Gaussian primitives for initialization^[23]. Meanwhile, we construct geometric confidence scores based on the cross-view consistency of depth predictions to guide the weighted initialization of the opacity and color of Gaussian primitives. In the third stage, we adopt a dynamic visibility regularization strategy during 3DGS training. This strategy reallocates

optimization gradients by randomly masking a subset of Gaussian primitives and compensating for the opacity of the remaining ones. It effectively alleviates overfitting to foreground regions, allowing distant and weakly visible regions to receive more sufficient gradient updates. Subsequent sections will provide detailed implementations of the contour-based Visual Hull baseline point cloud, the depth-guided point cloud enhancement, and the dynamic visibility regularization strategy.

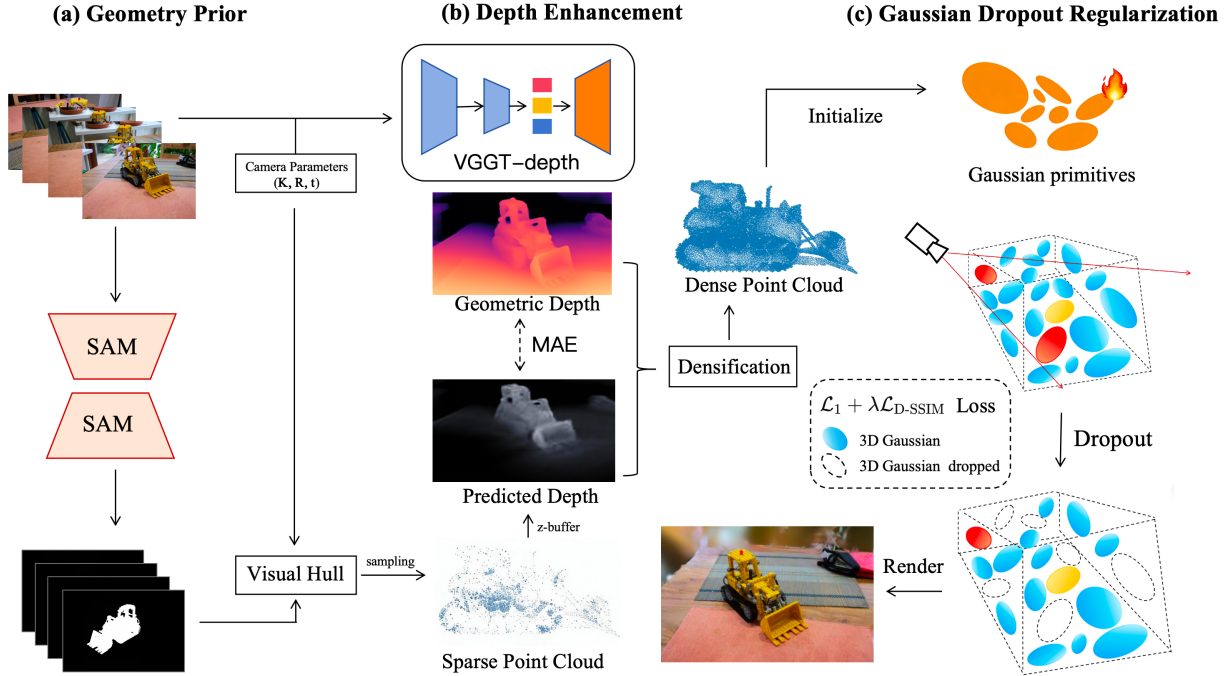


Fig. 1 Overview of SynGS. (a) Sparse-view images and masks are used to construct a visual hull for generating an initial point cloud prior. (b) VGGT-Depth enhances the incomplete geometry through depth-guided point cloud densification. (c) The densified point cloud is initialized as 3D Gaussian primitives and optimized with Gaussian Dropout regularization to improve sparse-view reconstruction robustness.

3.2 Structure-Prior-Guided Initialization

Under sparse-view conditions, particularly under sparse-input settings with 4, 6, or 9 views, the information available for 3D reconstruction is limited, and multi-view geometric constraints are highly restricted. Traditional Structure-from-Motion (SfM) ^[24] often recovers only sparse and incomplete point clouds, leading to geometric degradation. Since the 3D Gaussian Splatting (3DGS) pipeline typically uses SfM-calibrated camera parameters and sparse point clouds as initialization inputs, the density and spatial distribution of the initial point cloud directly influence the subsequent optimization stability and geometric convergence of Gaussian primitives ^[25]. To

address the lack of cross-view constraints in sparse-view reconstruction, which often results in numerous floating points and pseudo-geometry, we introduce a visual hull prior^[26] to generate a baseline initial point cloud. This prior guides Gaussian primitive initialization and provides more reliable geometric constraints. The visual hull is formed by the intersection of silhouette cones back-projected from multiple views, providing an approximation of the target’s geometric envelope and offering an additional shape prior when textures are insufficient or SfM points are limited.

Initialization Method based on Visual Hull. Let the number of input images be N , where $N \in [4, 9]$. For the i -th view, denote the input image as I_i , camera intrinsics as K_i , extrinsics as (R_i, t_i) , and the corresponding foreground mask as M_i . We assume that the camera pose for each view has been obtained via pre-calibration or SfM and aligned within a unified world coordinate system. Target masks are automatically generated by the Segment Anything Model (SAM), specifically the ViT-H version in our implementation. To mitigate the impact of boundary artifacts and local noise on subsequent Visual Hull construction, the initial masks undergo further processing, including largest connected component retention, hole filling, and a single morphological opening and closing. As a promptable general segmentation model, SAM is suitable for foreground mask acquisition under sparse multi-view conditions; however, its output may still contain local boundary errors. Therefore, we utilize it only as input for Visual Hull initialization rather than treating the mask itself as precise geometric supervision.

During the Visual Hull construction stage, we utilize the foreground masks and known camera parameters to back-project the 2D silhouette image into 3D silhouette cones and take their intersection to obtain a coarse Visual Hull region^[27]. For ease of implementation, we first use the axis-aligned bounding box (AABB) of this region as the sampling space and uniformly generate 1×10^5 candidate 3D points. If the number of retained points is insufficient for initialization, we continue supplementary sampling until the preset upper limit is reached or candidate points are exhausted. For any candidate point X , it is projected onto each reference view, and a mask consistency check is performed only on valid views where its projection falls within the image domain: the point is retained if and only if $\pi_i(x) \in M_i$ for all valid views i ; otherwise, it is discarded. The resulting point set constitutes the Visual Hull baseline point cloud. Compared to initialization methods that rely directly on extremely sparse SfM point clouds, this process does

not depend on cross-view image matching but instead utilizes silhouette consistency to provide geometric level geometric constraint for the distribution of subsequent Gaussian primitives. outlier removal. Since the Visual Hull is essentially a geometric envelope under silhouette constraints, floaters that are not sufficiently constrained by observation constraints may still appear in the initial point cloud when view coverage is limited, mask errors exist, or the target has a concave structure. These points can interfere with the spatial distribution of subsequent Gaussian initializations^[28]. Therefore, we perform a one-time outlier cleaning after generating the baseline point cloud rather than iteratively filtering during optimization. Specifically, we jointly employ Statistical Outlier Removal(SOR) and Radius Outlier Removal, (ROR) ^[29]. SOR calculates the average distance between each point and its K -nearest neighbors and removes outliers that significantly deviate from the mean based on the global distance distribution; we set $K = 20$ and the standard deviation multiplier threshold to 1.0. Then ROR removes isolated points based on local neighborhood density; we set the search radius to $r = 0.01d$, where d is the diagonal length of the AABB, and the minimum number of neighbors to $n_{\min} = 8$. Both filtering steps act directly on the initialization point cloud to obtain a baseline point set with a more compact spatial distribution and uniform local density for use in subsequent depth-guided point cloud enhancement and dynamic visibility regularization modules.

3.3 Multi-View Depth-Guided Point Cloud Densification

Under sparse-view conditions, the visual hull provides a stable geometric prior, but it only approximates the visible surface. Geometric gaps may remain in concave regions and fine external structures, limiting the geometric completeness and optimization stability of subsequent 3D Gaussian Splatting primitive initialization. To address this issue, we introduce a depth-guided point cloud densification mechanism based on the initial point cloud generated from the visual hull. To achieve reliable point supplementation, we incorporate the pre-trained Visual Geometry Grounded Transformer (VGGT) ^[30] as a multi-view consistent depth prior to guide point cloud densification and Gaussian attribute initialization.

Unlike pre-trained monocular depth models, such as Depth Anything V2 ^[31], which perform independent frame-by-frame inference, such models predict depth independently for each view and back-project these predictions into 3D space to generate candidate points. However, because these models target per-frame monocular depth estimation, they do not provide cross-view

geometric consistency guarantees. In sparse settings with 4–9 views, the cross-view depth consistency of these models drops significantly, producing noticeable hallucinatory depths at occlusion boundaries and low-texture regions, leading to point cloud drift and floating artifacts. In contrast, the VGGT method performs joint modeling of the input sparse views, outputting dense depths for each view under a shared scene geometric context. Therefore, we utilize its multi-view depth outputs as a geometric reference during the point supplementation stage to enhance geometric consistency across different views.

Inspired by the depth prediction concepts of VGGT, we design a depth prior adaptation branch specifically for the point cloud densification task, termed VGGT-Depth (as shown in Fig. 2). This branch utilizes the depth maps inferred by VGGT as an external geometric prior. It first encodes the depth information into learnable feature tokens and achieves explicit alignment between the depth prior and task features through a cross-attention mechanism^[32]. Subsequently, Vision Transformer layers are used to further model cross-view global contextual dependencies, enhancing the feature propagation and aggregation of geometric cues within the feature space. Finally, a Reassemble–Fusion multi-scale decoding structure is employed to perform progressive reassembly and fusion of features at different levels, generating a geometry-enhanced representation for the point cloud densification task. We perform hierarchical modeling of the cross-view consistency implicitly encoded in the depth prior across three levels: cross-frame context, ray-level projection consistency, and point-level 3D geometry recovery. The resulting geometric constraints are then subjected to explicit injection into the point cloud densification pipeline.

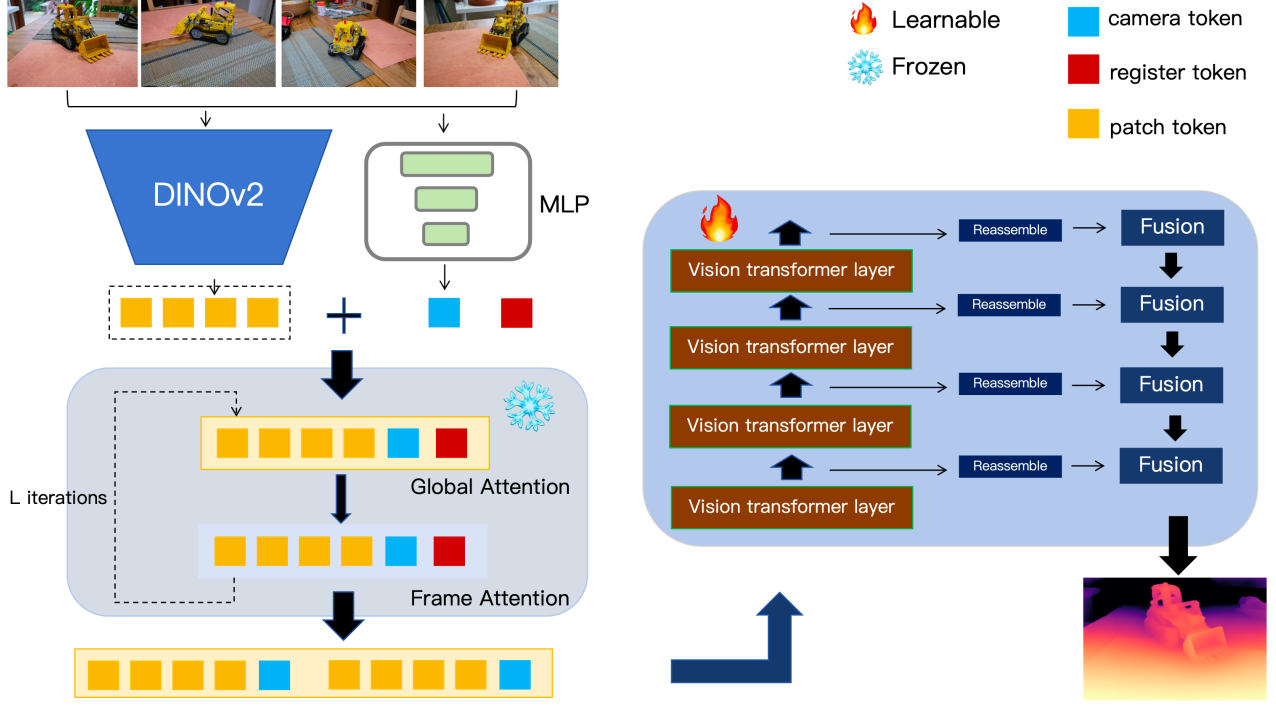


Fig. 2 Overview of the proposed VGGT-Depth architecture for depth-guided point cloud densification.

In the point cloud densification pipeline, we introduce VGGT-Depth as a pre-trained prior module, outputting depth maps and confidence scores for each view. This module is embedded into the densification process to filter high-quality views and reliable regions. Simultaneously, high-confidence depth maps are processed via back-projection to generate supplementary point clouds, filling in missing geometric regions in the initial visual hull point cloud. We further leverage depth consistency to guide the initial assignment of color and opacity, thereby reducing interference from low-confidence regions during the early optimization process.

To avoid the impact of scene scale variance on supplementation thresholds and voxel fusion, we first normalize the visual hull point cloud into a unified scene coordinate system, using the diagonal length L of the AABB^[33] as the scale baseline. All distance-related thresholds mentioned hereafter are set relative to L . We utilize only the depth prediction branch and its corresponding confidence maps output by VGGT, without introducing additional reconstruction heads or a field representation module. We explicitly exploit cross-view consistency information at three levels: frame-level for view quality screening, ray-level for reliable point supplementation determination, and point-level for color and opacity initialization. Specifically, given N input view images $\{I_i\}_{i=1}^N$ and their camera parameters $\{K_i, R_i, t_i\}_{i=1}^N$, we perform joint modeling by inputting all sparse views into the pre-trained VGGT-Depth to obtain dense depth predictions for each view. Based on

this, the point cloud densification pipeline consists of two stages: frame-level view selection and confidence-gated point supplementation. Using the cross-view consistent depth maps output by the pre-trained VGGT-Depth specialized for depth prediction, we implement a one-time point cloud densification on the visual hull point cloud, aiming to supplement geometric details and enhance initialization quality.

Frame-level View Selection. We use the depth model to perform single-frame inference for each frame, obtaining a per-frame depth map $\widehat{D}_i$. Subsequently, within the valid mask region, we calculate the Mean Absolute Error^[34] (MAE) between the inferred depth and the ground truth depth D_i provided in the dataset. Based on this, we define a view quality score q_v , which is used for subsequent frame selection:

$$q_v = 1 - \frac{\text{MAE}(\widehat{D}_i, D_i)}{\tau}, \quad (1)$$

where τ is the MAE normalization factor, which we set as the 95th percentile of the MAE distribution across all views in the validation set. A smaller error results in a higher q_v .

Considering that depth models are prone to hallucinations at occlusion boundaries, we use the ground truth depth as a reference for quality control. We apply a uniform threshold based on validation set statistics for filtering high-error views, retaining only the low-error view set for point supplementation. We set the frame-level selection threshold to $q_{th} = 0.4$. When $q_v < q_{th}$, the view is discarded. If the number of qualified views ($q_v \geq q_{th}$) is less than $0.8N$, the densification stage is skipped, and the original visual hull point cloud is used directly.

Confidence-Gated Point Supplementation. Each 3D point in the visual hull point cloud is projected along the ray onto the low-error views selected above. At the pixel projection location, three depth values are read: the geometric depth D_{geo} calculated from known camera poses $\{R_i, t_i\}$, the ground truth depth D_{gt} , and the predicted depth D_{pred} from VGGT-depth. We define an adaptive tolerance function as

$$\Delta(d) = \alpha \cdot d + \beta, \quad (2)$$

where d is the depth corresponding to the current pixel, $\alpha = 0.01$, and $\beta = 0.002L$. Only when $|D_{pred} - D_{gt}| < \Delta(D_{gt})$ and the geometric depth is substantially consistent with the ground truth depth is the ray determined to be a reliable observation. Otherwise, it is treated as an untrustworthy region caused by low-texture regions, depth estimation error, or occlusion, and is discarded.

Point supplementation is performed via back-projection only along reliable rays that pass the consistency test. For each point to be supplemented, we select the Top- K views with the highest consistency scores, where $K = 3$. In each view, sampling is performed within a small range along the ray relative to the predicted depth D_{pred} . We set a sampling radius $\delta = 0.01L$ and uniformly sample $M = 5$ candidate depth positions within the interval $[D_{\text{pred}} - \delta, D_{\text{pred}} + \delta]$ to generate candidate 3D points. Subsequently, two rounds of deduplication and uniformization are performed via voxel fusion. The first round uses a voxel side length $v_1 = 0.01L$ for coarse-grained coverage completion and duplicate candidate merging. The second round uses a voxel side length $v_2 = 0.004L$ for fine-grained uniformization and outlier removal, resulting in an enhanced point cloud p^* as a reliable geometric input for Gaussian primitive initialization.

Before optimizing 3D Gaussian Splatting parameters, the enhanced point cloud is explicitly mapped for Gaussian primitive initialization. For each 3D point X :

- Coordinates serve directly as the Gaussian centers μ .
- Scales s are initialized using local neighborhood adaptation: $k=8$ nearest neighbors are found, and the average distance is used as the initial isotropic radius.
- Rotation parameters are set as identity quaternions, with shapes and orientations learned during subsequent optimization.

The initial values for opacity o and Spherical Harmonic (SH) coefficients are set based on the cross-view consistency output by the pre-trained depth model. For a visible view i , the depth consistency weight is defined as:

$$w_i = C_i \cdot \exp\left(-\frac{|D_i^{\text{pred}} - D_i^{\text{gt}}|}{\Delta(d) + \varepsilon}\right) \cdot 1\left(|D_i^{\text{pred}} - D_i^{\text{gt}}| < \Delta(d)\right), \quad (3)$$

$$\widehat{w}_i = \frac{w_i}{\sum_{j \in \mathcal{V}} w_j + \varepsilon} \quad (4)$$

where C_i is the confidence output by VGGT-depth at that pixel, and $\varepsilon = 10^{-6}$ is a numerical stability term. This is then normalized across all visible views as $\widehat{w}_i$. where $1(\cdot)$ denotes the indicator function, which equals 1 if the condition holds and 0 otherwise.

Gaussian color SH coefficients attributes are initialized using multi-view weighted sampling. After projecting points onto visible views, pixel colors c_i are read via bilinear interpolation and fused according to normalized weights before being mapped to SH coefficients^[35]. This confidence-weighted approach effectively suppresses color bleeding at the occlusion boundary. Opacity is

initialized as the average of the cross-view consistency weights and truncated to the interval $[0.2, 0.8]$ to reduce the impact of low-confidence points on the early optimization process.

Compared to direct end-to-end point cloud generation, our proposed method aims to provide a more complete geometric and appearance initialization prior for 3D Gaussian Splatting in 4–9 view sparse scenes, thereby reducing geometric drift and floaters during early optimization.

3.4 *Densification Progressive Gaussian Dropout Regularization*

Under sparse-view settings, Gaussian primitives that are far from the camera or occluded are typically visible in only a few training views or pixels during optimization. In most iterations, these distant or occluded Gaussian primitives produce few effective projections over a long training period, resulting in extremely limited supervision from the reconstruction loss and small gradient magnitudes^[36]. As a result, parameter updates for attributes such as scale, color, and opacity are often very slow, making it difficult for these primitives to be sufficiently optimized. During training, effective optimization signals primarily come from a few visible regions^[37]. In the later stages of training, the model tends to overfit the training observations, degrading novel-view rendering quality and leading to artifacts.

To alleviate the aforementioned overfitting under sparse views, we propose a Dynamic Visibility Regularization (DVR) strategy to improve the gradient distribution and parameter updating of Gaussian primitives during the optimization process. Unlike introducing additional priors or network branches, the Dynamic Visibility Regularization strategy directly reconstructs the visibility and gradient distribution mechanism of the Gaussian primitives during training. Specifically, this strategy introduces random dropout regularization during the training phase. Inspired by the neural network dropout^[38] technique, we randomly mask a portion of the Gaussian primitives in each iteration to redistribute the gradient contribution in rendering synthesis, enabling the retained primitives to receive stronger gradient supervision and participate in parameter updates. As training progresses, this strategy gradually increases the dropout ratio to mitigate overfitting to the visible regions of training views in the later stages of training. The overall pipeline of this strategy is illustrated in Fig. 3.

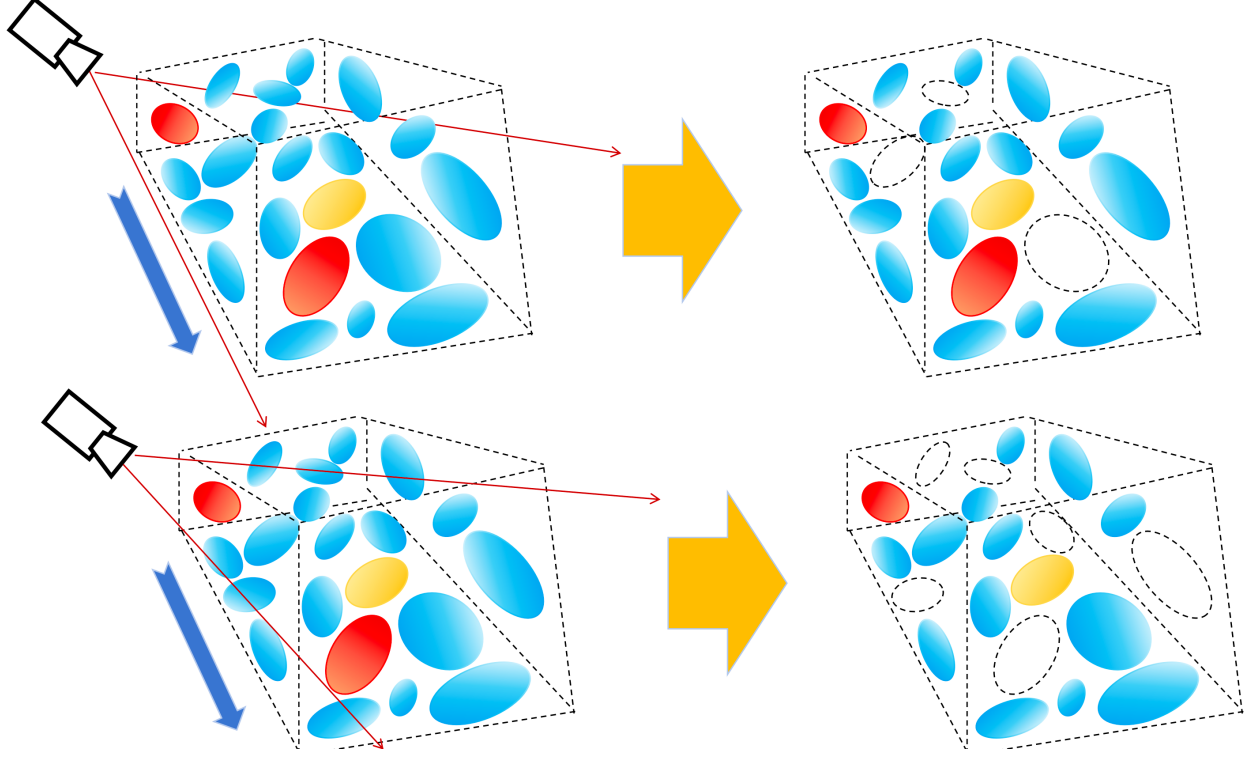


Fig. 3 Overview of the proposed Gaussian Dropout mechanism in Dynamic Visibility Regularization (DVR).

Linear Dropout Mechanism. During the training process, a portion of the Gaussian primitives is progressively and randomly masked. Unlike a constant dropout rate, a linear schedule can match the regularization strength with the training stage. In the early stages of training, a lower dropout rate is utilized to preserve geometry and alignment supervision, thereby avoiding underfitting. In the later stages of training, the dropout ratio is gradually increased to introduce stronger regularization constraints, which suppresses overfitting caused by view bias and improves novel view generalization. In terms of implementation, the linear dropout mechanism introduces a linear dropout rate schedule r_t that varies with the number of training steps (increasing linearly from 0) into the main training loop. Unlike permanent pruning, this mechanism employs random resampling at each iteration for the random masking of some Gaussian primitives. This allows temporarily masked Gaussian primitives to re-participate in rendering and backpropagation in subsequent iterations, thereby improving the imbalance of gradient updates among different Gaussian primitives.

We observed that random dropout under sparse views reduces the number of Gaussian primitives participating in pixel synthesis, leading to lower accumulated pixel opacity α . This weakens the occlusion capability of the foreground against the background, triggering artifacts such as

background leakage^[39]. Driven by the reconstruction loss, the optimization process compensates for this insufficient coverage—caused by the systemic rendering deviation introduced by dropout—by adjusting opacity and scale. This leads to overestimated scale and opacity, introducing additional parameter bias. To prevent these biases from accumulating during training, we adopt a lightweight Full-render Anchor method. Unlike DropGS^[40] which scales retained primitives to compensate for the expected rendering contribution, this method introduces additional full-render supervision to anchor to the complete rendering result without performing compensatory scaling. Specifically, we use a dropout rendering branch to provide regularization signals and additionally incorporate a full rendering branch^[41]. The rendering result of the full branch is similarly supervised by the color reconstruction loss (L1 + D-SSIM)^{[42] [43]}, ensuring the optimization target anchors on the full rendering without dropout, suppressing systematic bias, and stabilizing training.

Loss Function Design. We employ the classical color reconstruction loss function to optimize and train 3D Gaussian Splatting. This loss consists of an L1 term and a D-SSIM term, used to comprehensively measure the pixel-level error and structural similarity between the rendered image $\hat{I}$ and the ground truth image I :

$$\mathcal{L}_{\text{color}} = \mathcal{L}_1(\hat{I}, I) + \lambda \mathcal{L}_{\text{D-SSIM}}(\hat{I}, I), \quad (5)$$

where the balancing weight λ is set to 0.2 to balance the pixel accuracy of L1 and the structural similarity of D-SSIM. To alleviate overfitting and improve generalization performance under sparse views, we introduce a dropout regularization branch based on the standard color reconstruction loss. Specifically, at a given training iteration, Gaussian primitives are randomly masked according to a dropout rate to generate a binary mask $m_t = \{m_i \mid m_i \sim \text{Bernoulli}(1 - r_t)\}$. The completely masked primitives do not participate in the current rendering step, and the opacity of the remaining primitives is not compensated. The dropout rate increases progressively according to the following equation:

$$r_t = \min\left(r_{\text{max}}, \gamma \cdot \frac{t}{T}\right), \quad (6)$$

where the parameter (γ), which controls the growth slope is set within the range of 0.1 to 0.2 relative to the total number of training iterations, avoiding instability caused by the lack of compensation during training. Rendering is performed based on this mask to obtain the dropout branch image ($\widehat{I_{\text{drop}}} = \mathcal{R}(I; \Theta, m_t)$), while the complete rendered image is simultaneously

computed as the anchor ($\widehat{I}_{\text{full}} = \mathcal{R}(I; \Theta)$). The final training loss consists of the complete anchor term and the dropout regularization term:

$$\mathcal{L} = \mathcal{L}_{\text{color}}(\widehat{I}_{\text{full}}, I) + \beta \mathcal{L}_{\text{color}}(\widehat{I}_{\text{drop}}, I), \quad (7)$$

where the first term is the color reconstruction loss defined by Eq. (5), and the regularization weight for the dropout branch is typically set between 0.5 and 1.0.

By simultaneously constraining the complete rendering branch and the random dropout branch, this design ensures that the model maintains a consistent reconstruction target even when some Gaussian primitives are temporarily missing. The impact of the Dynamic Visibility Regularization strategy on the gradient distribution of Gaussian primitives across different distance intervals and novel view rendering performance will be further analyzed in the experimental section.

4 Results

4.1 Experimental Setup and Datasets

To evaluate the effectiveness of the proposed method, experiments were conducted on scenes from the Mip-NeRF 360 [44] and LLFF [45] datasets. Images in both datasets were downsampled by a factor of 8, and performance under sparse-view settings was evaluated on Mip-NeRF 360 using 4, 6, and 9 uniformly sampled input views. The evaluation metrics used were PSNR [46], SSIM [47], and LPIPS [48], which assess the quality of novel view rendering results from three perspectives: pixel-wise error, structural consistency, and perceptual similarity, respectively. These metrics are widely adopted in this field.

Sparse 360° Reconstruction Performance Evaluation. We compared our proposed method with several representative baselines, including the 3DGS baseline, few-shot NeRF methods (RegNeRF and SparseNeRF), and few-shot 3DGS methods (CoR-GS, DropGaussian, and D²GS). To ensure a fair comparison, the training and testing images within the same scene maintained a consistent resolution: the image resolution for Mip-NeRF 360 scenes was 779×520, and for the remaining scenes, it was 1280×720. In the initial optimization stage of our method, the 3DGS model was trained for 10,000 iterations, with floaters periodically removed every 500 iterations. Monocular depth was predicted by the ZoeDepth model. During the training phase, except for the key modules, all other parameter configurations remained consistent with 3DGS. The experiments covered

multiple scenes and were conducted for 30K iterations. All experiments were implemented based on the PyTorch framework. The hardware environment consisted of an 18 vCPU AMD EPYC 9754 128-Core Processor paired with an NVIDIA RTX 4090 GPU, and the Adam optimizer was used to train the model parameters. All quantitative evaluations and visualization results were obtained on this platform. The complete pipeline of DG-GS took approximately 10 minutes.

4.2 Quantitative and Qualitative Results

Under the same sparse input settings, we quantitatively compared the rendering quality of various methods on the 360° unbounded scenes of the Mip-NeRF 360 dataset, with the results shown in Table 1. The results in Table 1 demonstrates that under the 4/6/9-view settings, our method outperforms the second-best method, D²GS, in both PSNR and SSIM. Compared to D²GS, our PSNR improved by 20.4%, 12.9%, and 7.7%, respectively, and our SSIM improved by 2.7%, 1.7%, and 2.1%, respectively.

Table 1: Performance comparisons of sparse-view synthesis on the Mip-NeRF 360 dataset.

LPIPS* = LPIPS $\times$ 102 throughout this paper. The best and second-best entries are marked in red and blue respectively.

Methods	MipNeRF360 (4-view)			MipNeRF360 (6-view)			MipNeRF360 (9-view)		
	LPIPS* $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS* $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS* $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$
RegNeRF	19.88	12.59	0.841	20.72	13.41	0.847	19.70	13.68	0.852
SparseNeRF	17.76	12.83	0.845	19.74	13.42	0.901	21.56	14.36	0.823
3DGS	10.80	20.31	0.899	8.38	22.12	0.913	6.42	24.29	0.930
CoR-GS	11.76	20.45	0.712	9.28	22.57	0.886	7.04	24.73	0.916
DropGaussian	11.57	21.76	0.837	8.61	22.98	0.895	8.02	24.10	0.915
D ² GS	2.58	22.35	0.923	2.34	24.74	0.937	2.11	27.13	0.941
SynGS(Ours)	3.89	25.91	0.948	3.43	27.93	0.953	3.02	29.21	0.961

We compared SynGS with several NeRF-based methods (RegNeRF, FreeNeRF) and 3DGS-based methods (3DGS, CoR-GS, DropGaussian, D²GS) on the LLFF dataset. As shown in Table 2, SynGS consistently achieves the best performance. On the 4-view LLFF dataset at a 1/8 downsampling rate, SynGS achieved PSNR improvements of 4.12 dB, 3.47 dB, and 1.57 dB over DropGaussian, CoR-GS, and D²GS, respectively, while maintaining a consistent advantage in SSIM and LPIPS.

Table 2: Performance comparisons of sparse-view synthesis on the LLFF 360 dataset.

LPIPS* = LPIPS $\times$ 102 throughout this paper. The best and second-best entries are marked in red and blue respectively.

Methods	LLFF (4-view)			LLFF (6-view)			LLFF (9-view)		
	LPIPS*↓	PSNR↑	SSIM↑	LPIPS*↓	PSNR↑	SSIM↑	LPIPS*↓	PSNR↑	SSIM↑
RegNeRF	29.65	18.55	0.587	22.61	19.08	0.760	18.39	22.86	0.820
FreeNeRF	30.85	19.09	0.624	23.06	19.81	0.763	17.94	23.08	0.823
3DGS	22.93	19.32	0.649	13.45	23.80	0.814	9.68	25.44	0.860
CoR-GS	19.66	20.45	0.696	12.53	23.87	0.840	8.94	26.70	0.874
DropGaussian	21.93	19.80	0.621	13.79	23.41	0.803	9.40	25.87	0.868
D ² GS	17.98	22.35	0.746	13.85	23.91	0.846	8.91	26.80	0.871
SynGS(Ours)	16.92	23.92	0.762	12.54	25.17	0.848	8.96	26.71	0.871

To further analyze the impact of Dynamic Visibility Regularization on the training process, we calculated the average gradient magnitude of Gaussian primitives across different normalized distance intervals, with the results shown in Fig. 4. The horizontal axis represents the normalized distance interval from the camera to the scene, and the vertical axis represents the average gradient magnitude of the corresponding Gaussian primitives. Under sparse-view settings, our method obtained higher average gradient magnitudes for Gaussian points across all distance intervals. The improvement is more pronounced in the long-distance intervals, where the average gradient increased from approximately 0.2 to approximately 1.2 in the 0.8–1.0 interval.

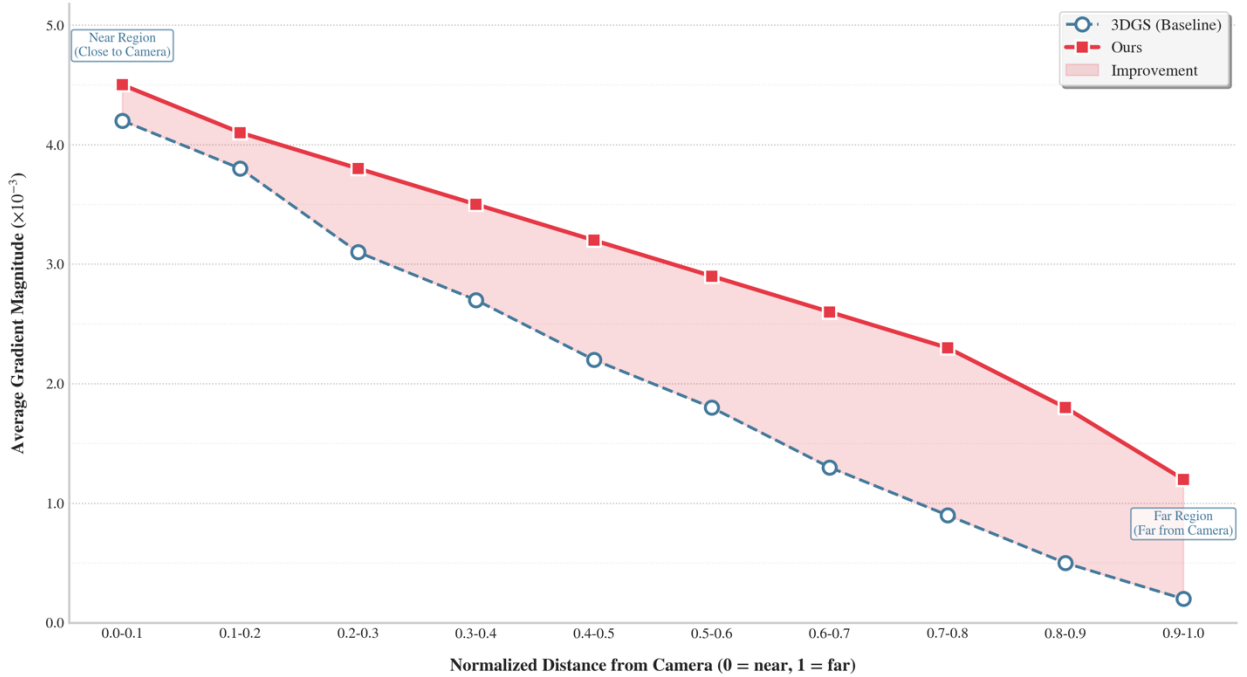


Fig. 4: Average gradient magnitude of Gaussian primitives across normalized distance intervals.

Figure. 5 illustrates the qualitative rendering comparison of various methods on different datasets under the 4-view sparse input condition. Compared to the baseline methods, our approach exhibits fewer artifacts in several scenes and presents more complete structural boundaries.



Fig. 5: Comparison of novel view rendering results on the Mip-NeRF 360 dataset.

4.3 Ablation Study

To verify the effectiveness of the design of each module in our method, we conducted ablation studies on these modules. We evaluated the specific contributions of the three modules: Visual Hull Init, Depth Densification, and Visibility Balancing. During the experiments, the training parameters and hyperparameter settings were kept consistent, and comparisons were made solely by adding or removing different modules to ensure the comparability and reliability of the conclusions.

Table 3: Impact of different modules on performance

Visual Hull Init	Depth Densification	Visibility Balancing	SSIM↑	PSNR↑	LPIPS↓
✓			0.865	17.51	1.128
✓	✓		0.902	24.96	0.879
✓	✓	✓	0.943	26.18	0.609

Note: ✓ indicates the corresponding module is enabled; ↑ indicates higher is better; ↓ indicates lower is better; bold data indicates the best values.

Table 3 presents the quantitative results under different module combinations. As the Visual Hull Init, Depth Densification, and Visibility Balancing modules are progressively added, PSNR, SSIM, and LPIPS exhibit varying degrees of change; the configuration containing all modules achieves the optimal results. Observing the magnitude of metric changes, the performance improvement is

more significant after incorporating Depth Densification. This quantitative analysis validates the overall performance gain brought by the joint use of the three modules.

5 Discussion and Conclusion

Targeting sparse-view 3D reconstruction, we propose a Gaussian splatting optimization framework that incorporates a geometric prior. By performing geometric initialization, depth-consistency-driven back-projection for point cloud generation, subsequent point cloud densification, and a Densification-Progressive Gaussian Dropout Regularization strategy, the proposed framework improves representation quality and training stability under sparse-input conditions. Experimental results on unbounded 360° scenes demonstrate that the proposed method consistently improves novel view synthesis quality under sparse-view settings and reduces view dependence to a certain extent. Further analysis based on both quantitative and qualitative results shows that the method achieves relatively consistent advantages in pixel-level fidelity and structural consistency. However, in terms of perceptual quality, the LPIPS score of our method is slightly worse than that of D2GS, indicating that although the method maintains high PSNR and SSIM performance, there remains room for further improvement in perceptual similarity.

From the perspective of the training mechanism, Densification-Progressive Gaussian Dropout Regularization contributes to the improvement in reconstruction quality. During optimization, it improves gradient allocation for low-visibility Gaussian primitives, enabling distant regions to receive more sufficient supervision signals during training and effectively alleviating the overfitting problem caused by low visibility. This phenomenon is consistent with the observed trend of average gradients across different normalized distance intervals, demonstrating the effectiveness of the proposed method in scene geometry and appearance modeling under sparse-view conditions.

Based on the ablation study results, all three key modules contribute positively to the final performance. Among them, depth-consistency-based densification yields a more significant performance improvement, indicating that, under sparse-input conditions, depth-prior-guided back-projection-based point supplementation can effectively address insufficient scene geometry coverage. Visual hull initialization provides a more reliable geometric initialization for subsequent optimization, allowing the model to obtain a more stable point cloud distribution in the early stage. Meanwhile, Densification-Progressive Gaussian Dropout Regularization further improves

gradient balance across different spatial regions, enhancing the overall reconstruction quality. The best results are achieved when all three modules are used jointly, confirming the complementarity of the modules within the proposed framework.

Although the proposed method demonstrates advantages in rendering quality and geometric consistency, it still has limitations in terms of memory overhead and computational efficiency. Specifically, the point cloud densification process introduces additional memory costs, and training efficiency remains to be improved compared with existing highly efficient 3DGS methods. In future work, it will be necessary to explore more compact Gaussian parameterizations and scene representations, in combination with dynamic Gaussian pruning or hierarchical Gaussian representations, to alleviate the redundancy of primitives introduced by densification. This may further improve training and inference efficiency while maintaining rendering quality.

Biographies

Junjie Geng received the Ph.D. degree from the Communication University of China, Beijing, China. His research interests include intelligent media technology and applications.

Ao Zhang is currently pursuing the master's degree at the Communication University of China, Beijing, China. His research interests include intelligent media technology and applications.

Lian Duan is currently pursuing the master's degree at the Communication University of China, Beijing, China. Her research interests include audiovisual new media.

Disclosures

The authors declare no conflicts of interest.

Code and Data

The code and data supporting the findings of this study are available from the corresponding author upon reasonable request.

Declaration of Generative AI and AI-assisted technologies in the writing process

During the preparation of this manuscript, the authors used Gemini 3.1 Pro (Google) to assist with language refinement, content organization, and writing improvement. All AI-assisted content was

carefully reviewed, revised, and validated by the authors. The authors take full responsibility for the accuracy, integrity, and originality of the final manuscript.

References

1. J. Gao et al., “Advances in differentiable rendering based on three-dimensional Gaussian splatting (invited),” *Laser & Optoelectronics Progress* 61(16), 1611010 (2024).
2. J. Yang, M. Pavone, and Y. Wang, “FreeNeRF: Improving few-shot neural rendering with free frequency regularization,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 8254-8263 (2023).
3. H. Tang, “Research on three-dimensional reconstruction methods for sparse-view scenes based on neural radiance fields,” PhD Dissertation, Chongqing University of Technology (2025) [doi:10.27753/d.cnki.gcqgx.2025.001387].
4. R. Li and C. Yang, “Research on three-dimensional reconstruction technology based on improved SIFT algorithm,” *Journal of Chongqing Technology and Business University (Natural Science Edition)* 42(1), 105-111 (2025).
5. J. Chung, J. Oh, and K. M. Lee, “Depth-regularized optimization for 3D Gaussian splatting in few-shot images,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops*, 811-820 (2024).
6. B. Mildenhall et al., “NeRF: representing scenes as neural radiance fields for view synthesis,” *Communications of the ACM* 65(1), 99-106 (2021) [doi:10.1145/3503250].
7. G. Wang et al., “SparseNeRF: Distilling depth ranking for few-shot novel view synthesis,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 9031-9042 (2023).
8. K. Deng et al., “Depth-Supervised NeRF: Fewer views and faster training for free,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 12872-12881 (2022).
9. M. Niemeyer et al., “RegNeRF: Regularizing neural radiance fields for view synthesis from sparse inputs,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 5480-5490 (2022).
10. B. Kerbl et al., “3D Gaussian Splatting for real-time radiance field rendering,” *ACM Transactions on Graphics (TOG)* 42(4), 139 (2023).
11. J. Chung, J. Oh, and K. M. Lee, “Depth-regularized optimization for 3D Gaussian splatting in few-shot images,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops*, 811-820 (2024).

12. J. Li et al., “DNGaussian: Optimizing sparse-view 3D Gaussian radiance fields with global-local depth normalization,” in Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 20775-20785 (2024).
13. Z. H. Zhu et al., “FSGS: Real-time few-shot view synthesis using Gaussian splatting,” in Proceedings of the European Conference on Computer Vision (ECCV), 145-163 (2024).
14. H. L. Xiong et al., “SparseGS: Real-time 360° sparse view synthesis using Gaussian splatting,” arXiv:2312.00206 (2024).
15. S. S. Huang et al., “SC-NeuS: Consistent neural surface reconstruction from sparse and noisy views,” in Proceedings of the AAAI Conference on Artificial Intelligence (2024).
16. Z. X. Zou et al., “Sparse3D: Distilling multiview-consistent diffusion for object reconstruction from sparse views,” in Proceedings of the AAAI Conference on Artificial Intelligence (2024).
17. Y. X. Xiao et al., “Level-S2fM: Structure from motion on neural level set of implicit surfaces,” in Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 17205-17214 (2023).
18. K. Li et al., “Simple non-destructive and 3D multi-layer visual hull reconstruction with an ultrabroadband carbon nanotubes photo-imager,” Advanced Optical Materials 12(14), 2302847 (2024).
19. Z. Yang et al., “Real-time light-field generation based on the visual hull for the 3D light-field display with free-viewpoint texture mapping,” Optics Express 31(2), 1125-1140 (2023).
20. A. I. Lysykh, D. D. Zhdanov, and M. I. Sorokin, “Method of geometry reconstruction from a set of RGB images using differentiable rendering and visual hull,” Programming and Computer Software 51(3), 150-159 (2025).
21. A. Kirillov et al., “Segment anything,” in Proceedings of the IEEE/CVF International Conference on Computer Vision, 4015-4026 (2023).
22. H. Balta et al., “Fast statistical outlier removal based method for large 3D point clouds of outdoor environments,” IFAC-PapersOnLine 51(22), 348-353 (2018).
23. L. Yang et al., “Depth Anything V2,” Advances in Neural Information Processing Systems 37, 21875-21911 (2024).
24. J. Wang et al., “VGGT: Visual geometry grounded transformer,” in Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 5294-5306 (2025).
25. A. P. A. Ama and A. M. N. Tampah, “Availability of assets and livelihood strategies of school dropout teenage mothers in Adaklu District, Ghana,” Cogent Social Sciences 9(1) (2023).
26. Y. Xu et al., “DropoutGS: Dropping out Gaussians for better sparse-view rendering,” in Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 701-710 (2025).

27. J. T. Barron et al., “Mip-NeRF 360: Unbounded anti-aliased neural radiance fields,” in Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 5460-5469 (2022).
28. B. Mildenhall et al., “Local light field fusion: Practical view synthesis with prescriptive sampling guidelines,” ACM Transactions on Graphics (TOG) 38(4), 29 (2019).
29. Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, “Image quality assessment: From error visibility to structural similarity,” IEEE Transactions on Image Processing 13(4), 600-612 (2004).
30. R. Zhang et al., “The unreasonable effectiveness of deep features as a perceptual metric,” in Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 586-595 (2018).
31. Z. Ding, C. T. Lee, M. Zhu, et al., “EyeNavGS: A 6-DoF navigation dataset and record-n-replay software for real-world 3DGS scenes in VR,” in Proceedings of the 33rd ACM International Conference on Multimedia, 13126-13132 (2025).
32. T. Wang, M. Li, G. Zeng, et al., “Gaussian herding across pens: An optimal transport perspective on global Gaussian reduction for 3DGS,” Advances in Neural Information Processing Systems 38, 157898-157923 (2026).
33. Y. Kim, J. Choi, and S. Yoon, “GDoFS: Gaussian DoF Separation for Plausible 3D Geometry in Sparse-View 3DGS,” in Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision, 6643-6652 (2026).
34. R. Van De Ven, T. Bresilla, B. Nelissen, et al., “Enhancing annotations for 5D apple pose estimation through 3D Gaussian Splatting (3DGS),” Smart Agricultural Technology, 101910 (2026).
35. J. Zhang, J. Jiang, Y. Chen, et al., “Cob-GS: Clear object boundaries in 3DGS segmentation based on boundary-adaptive Gaussian splitting,” in Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 19335-19344 (2025).
36. K. Luo, S. Ye, T. Ouyang, et al., “MEGA: Mesh-Aligned 3DGS Towards Geometry-Preserving Online Reconstruction,” in IFIP International Conference on Network and Parallel Computing, 396-408 (2024).
37. J. Dong and Y. X. Wang, “3DGS-Drag: Dragging Gaussians for intuitive point-based 3D editing,” in International Conference on Learning Representations, 82799-82815 (2025).
38. J. Huang, S. Xu, H. Yu, et al., “GSDeformer: Direct, real-time and extensible cage-based deformation for 3D Gaussian splatting,” arXiv preprint arXiv:2405.15491 (2024).
39. X. Lin, S. Luo, X. Shan, et al., “HQGS: High-quality novel view synthesis with Gaussian splatting in degraded scenes,” in The Thirteenth International Conference on Learning Representations (2025).

40. D. Shi, S. Cao, L. Fan, et al., “TrackGS: Optimizing COLMAP-free 3D Gaussian Splatting with global track constraints,” in *Proceedings of the AAAI Conference on Artificial Intelligence* 40(11), 8960-8968 (2026).
41. Y. Zhang, Z. Wang, J. Han, et al., “GS-Net: Generalizable Plug-and-Play 3D Gaussian Splatting Module,” *arXiv preprint arXiv:2409.11307* (2024).
42. B. Xiong, N. Zheng, J. Liu, et al., “GauU-Scene V2: Assessing the reliability of image-based metrics with expansive LiDAR image dataset using 3DGS and NeRF,” *arXiv preprint arXiv:2404.04880* (2024).
43. J. Byun et al., “CrossGaussian: Enhancing remote collaboration through 3D Gaussian Splatting and real-time 360 streaming,” in *Adjunct Proceedings of the 38th Annual ACM Symposium on User Interface Software and Technology*, 1-3 (2025).
44. S. Lee et al., “EmbodiedSplat: Online feed-forward semantic 3DGS for open-vocabulary 3D scene understanding,” *arXiv preprint arXiv:2603.04254* (2026).
45. B. Kathariya et al., “Gaussian Splatting: State-of-the-arts and future trends,” in *2025 International Conference on Computing, Networking and Communications*, 876-881 (2025).
46. S. Li et al., “GauRast: Enhancing GPU triangle rasterizers to accelerate 3D Gaussian splatting,” in *2025 62nd ACM/IEEE Design Automation Conference*, 1-7 (2025).
47. Y. Shi et al., “LapisGS: Layered progressive 3D Gaussian splatting for adaptive streaming,” in *2025 International Conference on 3D Vision*, 991-1000 (2025).
48. W. Zhang et al., “Glossy-Gaussian: Adaptive anisotropic Gaussians for view-dependent appearances,” *IEEE Signal Processing Letters* (2025).